

Documento dei Requisiti

Progetto: Gestione Prestiti di una Biblioteca Personale

Corso:	Informatica / Programmazione JavaScript
Scadenza:	29 Aprile 2026
Versione:	2.0
Data documento:	28 Aprile 2026

1. Introduzione

Il presente documento descrive i requisiti funzionali e non funzionali dell'applicazione **Gestione Prestiti di una Biblioteca Personale**, sviluppata in JavaScript. L'obiettivo del sistema è di consentire all'utente di gestire i libri della propria collezione privata, monitorarne la disponibilità e tracciare i movimenti di prestito e restituzione.

2. Requisiti Funzionali

I requisiti funzionali descrivono le operazioni che il sistema deve essere in grado di eseguire. Sono classificati in base alla versione di implementazione.

2.1 Versione Base

ID	Requisito	Descrizione	Priorità
RF01	Registrazione libro	Il sistema deve permettere l'aggiunta di un nuovo libro con codice identificativo univoco, titolo e autore.	Alta
RF02	Prestito libro	Il sistema deve associare un libro a un lettore registrando nome del lettore e data del prestito. Il libro cambia stato in 'in prestito'.	Alta
RF03	Restituzione libro	Il sistema deve aggiornare lo stato di un libro da 'in prestito' a 'disponibile' e registrare la data di restituzione.	Alta

2.2 Versione Avanzata

ID	Requisito	Descrizione	Priorità
RF04	Filtra disponibili	Il sistema deve permettere di visualizzare solo i libri con stato 'disponibile'.	Media
RF05	Filtra in prestito	Il sistema deve permettere di visualizzare solo i libri attualmente in prestito.	Media
RF06	Gestione ritardi	Il sistema deve evidenziare i prestiti che superano un numero configurabile di giorni (default: 30).	Media

RF07	Storico operazioni	Il sistema deve conservare la cronologia completa di tutte le operazioni: aggiunte, prestiti e restituzioni con data e ora.	Alta
-------------	---------------------------	---	------

3. Requisiti Non Funzionali

I requisiti non funzionali definiscono le qualità e i vincoli del sistema, indipendentemente dalle funzionalità specifiche.

ID	Categoria	Descrizione
RNF 01	Modularità	Il codice è organizzato in moduli separati (biblioteca.js, main.js) per garantire la separazione delle responsabilità.
RNF 02	Manutenibilità	Ogni classe e metodo è documentato con commenti JSDoc completi (descrizione, @param, @returns, @throws).
RNF 03	Affidabilità	Il sistema gestisce correttamente i casi di errore (libro non trovato, libro già in prestito) tramite eccezioni appropriate.
RNF 04	Leggibilità	Il codice segue convenzioni di naming chiare: camelCase per metodi e variabili, PascalCase per le classi.
RNF 05	Portabilità	Il sistema funziona su qualsiasi piattaforma con Node.js installato, senza dipendenze esterne.
RNF 06	Versionamento	Il progetto è gestito con Git, con commit regolari e descrittivi che documentano l'avanzamento del lavoro.
RNF 07	Scalabilità	L'architettura a classi permette l'aggiunta futura di nuove funzionalità senza ristrutturare il codice esistente.

4. Architettura del Sistema

Il sistema adotta un'architettura **Object-Oriented** basata su classi. Il codice è organizzato in due file principali:

File	Responsabilità
biblioteca.js	Contiene le classi Libro, Prestito, Storico e Biblioteca con tutte le funzionalità del sistema.
main.js	Punto di ingresso: istanzia la Biblioteca e dimostra tutte le funzionalità disponibili.

5. Materiale Consegnato

#	File / Artefatto	Descrizione
1	UML_CasiDUso.png	Diagramma UML dei casi d'uso del sistema.
2	Requisiti_Biblioteca.pdf	Questo documento: requisiti funzionali e non funzionali.

3	biblioteca.js	Classi Libro, Prestito, Storico e Biblioteca con documentazione JSDoc.
4	main.js	Punto di ingresso e demo completa dell'applicazione.
5	link.txt	URL del repository GitHub pubblico.